```
from flask import Flask, render_template, jsonify, request
import os
import pandas as pd
import random
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
import joblib

app = Flask(__name__)

DATA_FILE = "sensor_data.csv"
MODEL_FILE = "irrigation_model.pkl"

def generate_seed_data():
    rows = []
    for i in range(200):
        soil_moisture = random.randint(10, 80)
        temperature = random.randint(18, 42)
        humidity = random.randint(20, 90)
        rainfall = random.randint(0, 30)
        crop_stage = random.randint(1, 5)
        needs_irrigation = 1 if (soil_moisture < 35 and temperature > 28 and humidity < 55) else 0

        if rainfall > 20:
            needs_irrigation = 0

        rows.append({
            "soil_moisture": soil_moisture,
            "temperature": temperature,
            "humidity": humidity,
            "rainfall": rainfall,
            "crop_stage": crop_stage,
            "needs_irrigation": needs_irrigation
        })

    df = pd.DataFrame(rows)
    df.to_csv(DATA_FILE, index=False)

def train_model():
    if not os.path.exists(DATA_FILE):
        generate_seed_data()

    df = pd.read_csv(DATA_FILE)

    features = ["soil_moisture", "temperature", "humidity", "rainfall", "crop_stage"]
    target = "needs_irrigation"

    X = df[features]
    y = df[target]

    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.2, random_state=42, stratify=y
    )

    model = RandomForestClassifier(n_estimators=100, random_state=42)
    model.fit(X_train, y_train)

    preds = model.predict(X_test)
    acc = accuracy_score(y_test, preds)
```

Edit with WPS Office

```
    print(f"Model accuracy: {acc:.2f}")

    joblib.dump(model, MODEL_FILE)
    return model

def load_or_train_model():
    if os.path.exists(MODEL_FILE):
        return joblib.load(MODEL_FILE)
    return train_model()

model = load_or_train_model()

@app.route("/")
def index():
    return render_template("index.html")

@app.route("/api/health")
def health():
    return jsonify({"status": "ok"})

@app.route("/api/sensors")
def sensors():
    if not os.path.exists(DATA_FILE):
        generate_seed_data()

    df = pd.read_csv(DATA_FILE).tail(10)
    return jsonify(df.to_dict(orient="records"))

@app.route("/api/predict", methods=["POST"])
def predict():
    data = request.get_json()
    sample = {
        "soil_moisture": float(data.get("soil_moisture", 30)),
        "temperature": float(data.get("temperature", 28)),
        "humidity": float(data.get("humidity", 50)),
        "rainfall": float(data.get("rainfall", 5)),
        "crop_stage": int(data.get("crop_stage", 2))
    }

    prediction = model.predict(pd.DataFrame([sample]))[0]
    probability = model.predict_proba(pd.DataFrame([sample]))[0]
    confidence = float(max(probability))

    result = {
        "needs_irrigation": bool(prediction),
        "confidence": round(confidence, 2),
        "input": sample
    }
    return jsonify(result)

if __name__ == "__main__":
    app.run(debug=True, host="0.0.0.0", port=5000)
    <!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Smart Agriculture AI</title>
    <style>
        body {
```

Edit with WPS Office

```
        font-family: Arial, sans-serif;
        margin: 30px;
        background: #f4f7f4;
        color: #1c2b1c;
}
.container {
        max-width: 1000px;
        margin: auto;
}
.cards {
        display: grid;
        grid-template-columns: repeat(auto-fit, minmax(180px, 1fr));
        gap: 16px;
        margin-bottom: 25px;
}
.card {
        background: white;
        padding: 18px;
        border-radius: 10px;
        box-shadow: 0 2px 8px rgba(0,0,0,0.08);
}
.card h3 { margin: 0 0 10px; font-size: 16px; }
.card p { margin: 0; font-size: 24px; font-weight: bold; }
form {
        background: white;
        padding: 20px;
        border-radius: 12px;
        box-shadow: 0 2px 8px rgba(0,0,0,0.08);
        margin-bottom: 25px;
}
.grid {
        display: grid;
        grid-template-columns: repeat(auto-fit, minmax(180px, 1fr));
        gap: 16px;
}
label {
        display: block;
        font-size: 14px;
        margin-bottom: 8px;
}
input, select {
        width: 100%;
        padding: 10px;
        box-sizing: border-box;
        border: 1px solid #c9d2c9;
        border-radius: 8px;
        font-size: 14px;
}
button {
        margin-top: 18px;
        padding: 12px 16px;
        background: #2e7d32;
        color: white;
        border: none;
        border-radius: 8px;
        cursor: pointer;
        font-size: 15px;
}
#result {
        margin-top: 20px;
```

Edit with WPS Office

```
            background: #eaf7ea;
            border: 1px solid #b9dcb9;
            border-radius: 8px;
            padding: 16px;
            font-size: 16px;
        }
        table {
            width: 100%;
            border-collapse: collapse;
            background: white;
            border-radius: 8px;
            overflow: hidden;
        }
        th, td {
            padding: 10px 12px;
            border-bottom: 1px solid #e5e5e5;
            text-align: left;
        }
    </style>
</head>
<body>
    <div class="container">
        <h1>AI Smart Agriculture Dashboard</h1>

        <div class="cards">
            <div class="card">
                <h3>Soil Moisture</h3>
                <p id="soilValue">--</p>
            </div>
            <div class="card">
                <h3>Temperature</h3>
                <p id="tempValue">--</p>
            </div>
            <div class="card">
                <h3>Humidity</h3>
                <p id="humidityValue">--</p>
            </div>
            <div class="card">
                <h3>Rainfall</h3>
                <p id="rainValue">--</p>
            </div>
        </div>

        <form id="predictForm">
            <h2>Predict irrigation need</h2>
            <div class="grid">
                <div>
                    <label>Soil Moisture (%)</label>
                    <input id="soil_moisture" type="number" value="32" min="0" max="100">
                </div>
                <div>
                    <label>Temperature (捞 C)</label>
                    <input id="temperature" type="number" value="30" min="0" max="60">
                </div>
                <div>
                    <label>Humidity (%)</label>
                    <input id="humidity" type="number" value="45" min="0" max="100">
                </div>
                <div>
                    <label>Rainfall (mm)</label>
```

Edit with WPS Office

```
                <input id="rainfall" type="number" value="5" min="0" max="100">
            </div>
            <div>
                <label>Crop Stage</label>
                <select id="crop_stage">
                    <option value="1">1 - Early growth</option>
                    <option value="2" selected>2 - Vegetative</option>
                    <option value="3">3 - Flowering</option>
                    <option value="4">4 - Fruiting</option>
                    <option value="5">5 - Harvest</option>
                </select>
            </div>
        </div>

        <button type="submit">Analyze</button>
    </form>

    <div id="result">Prediction result will appear here.</div>

    <h2>Recent sensor readings</h2>
    <table>
        <thead>
            <tr>
                <th>Soil</th>
                <th>Temp</th>
                <th>Humidity</th>
                <th>Rain</th>
                <th>Crop</th>
                <th>Irrigation</th>
            </tr>
        </thead>
        <tbody id="sensorTableBody"></tbody>
    </table>
</div>

<script>
    async function loadSensors() {
        const res = await fetch("/api/sensors");
        const rows = await res.json();

        const tableBody = document.getElementById("sensorTableBody");
        tableBody.innerHTML = "";

        rows.forEach(row => {
            const tr = document.createElement("tr");
            tr.innerHTML = `
                <td>${row.soil_moisture}</td>
                <td>${row.temperature}</td>
                <td>${row.humidity}</td>
                <td>${row.rainfall}</td>
                <td>${row.crop_stage}</td>
                <td>${row.needs_irrigation ? "Yes" : "No"}</td>
            `;
            tableBody.appendChild(tr);
        });

        if (rows.length > 0) {
            const latest = rows[rows.length - 1];
            document.getElementById("soilValue").textContent = latest.soil_moisture + "%";
            document.getElementById("tempValue").textContent = latest.temperature + "捞
```

Edit with WPS Office

```
C";
                document.getElementById("humidityValue").textContent = latest.humidity + "%";
                document.getElementById("rainValue").textContent = latest.rainfall + " mm";
            }
        }

        document.getElementById("predictForm").addEventListener("submit", async function (e) {
            e.preventDefault();

            const payload = {
                soil_moisture: Number(document.getElementById("soil_moisture").value),
                temperature: Number(document.getElementById("temperature").value),
                humidity: Number(document.getElementById("humidity").value),
                rainfall: Number(document.getElementById("rainfall").value),
                crop_stage: Number(document.getElementById("crop_stage").value)
            };

            const res = await fetch("/api/predict", {
                method: "POST",
                headers: { "Content-Type": "application/json" },
                body: JSON.stringify(payload)
            });

            const data = await res.json();
            const status = data.needs_irrigation ? "IRRIGATION REQUIRED" : "NO IRRIGATION
NEEDED";

            document.getElementById("result").innerHTML = `
                <strong>${status}</strong><br>
                Confidence: ${(data.confidence * 100).toFixed(1)}%<br>
                Soil moisture: ${data.input.soil_moisture}%<br>
                Temperature: ${data.input.temperature}掳 C
            `;
        });

        loadSensors();
        setInterval(loadSensors, 3000);
    </script>
</body>
</html>
from flask import Flask, render_template, jsonify, request
import os
import pandas as pd
import random
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
import joblib

app = Flask(__name__)

DATA_FILE = "sensor_data.csv"
MODEL_FILE = "irrigation_model.pkl"

def generate_seed_data():
    rows = []
    for i in range(300):
        soil_moisture = random.randint(10, 80)
        temperature = random.randint(18, 42)
        humidity = random.randint(20, 95)
```

Edit with WPS Office

```
            rainfall = random.randint(0, 35)
            crop_stage = random.randint(1, 5)

            needs_irrigation = 1 if (
                soil_moisture < 35 and
                temperature > 28 and
                humidity < 55
            ) else 0

            if rainfall >= 15:
                needs_irrigation = 0

            rows.append({
                "soil_moisture": soil_moisture,
                "temperature": temperature,
                "humidity": humidity,
                "rainfall": rainfall,
                "crop_stage": crop_stage,
                "needs_irrigation": needs_irrigation
            })

    df = pd.DataFrame(rows)
    df.to_csv(DATA_FILE, index=False)

def train_model():
    if not os.path.exists(DATA_FILE):
        generate_seed_data()

    df = pd.read_csv(DATA_FILE)

    features = ["soil_moisture", "temperature", "humidity", "rainfall", "crop_stage"]
    target = "needs_irrigation"

    X = df[features]
    y = df[target]

    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.2, random_state=42, stratify=y
    )

    model = RandomForestClassifier(n_estimators=200, random_state=42)
    model.fit(X_train, y_train)

    preds = model.predict(X_test)
    acc = accuracy_score(y_test, preds)
    print(f"Model accuracy: {acc:.2f}")

    joblib.dump(model, MODEL_FILE)
    return model

def load_or_train_model():
    if os.path.exists(MODEL_FILE):
        return joblib.load(MODEL_FILE)
    return train_model()

def calculate_irrigation_plan(sample):
    soil = sample["soil_moisture"]
    temp = sample["temperature"]
    humidity = sample["humidity"]
    rainfall = sample["rainfall"]
```

Edit with WPS Office

```
    crop_stage = int(sample["crop_stage"])

    crop_factor = {1: 0.8, 2: 1.0, 3: 1.2, 4: 1.1, 5: 0.9}.get(crop_stage, 1.0)

    if rainfall >= 12:
        return {
            "needs_irrigation": False,
            "recommended_liters": 0.0,
            "reason": "Rainfall is sufficient, so irrigation is skipped to reduce water waste."
        }

    if soil > 55 and humidity > 65:
        return {
            "needs_irrigation": False,
            "recommended_liters": 0.0,
            "reason": "Soil moisture and humidity are already adequate."
        }

    deficite = max(0, 50 - soil)
    temp_load = max(0, temp - 25) * 0.7
    humid_load = max(0, 65 - humidity) * 0.25

    water_liters = (deficite * 1.6 + temp_load + humid_load) * crop_factor

    if rainfall > 0:
        water_liters *= 0.6

    water_liters = max(0.0, min(water_liters, 65.0))
    needs = water_liters > 2.0

    if needs:
        reason = "Targeted irrigation recommended to avoid runoff and overwatering."
    else:
        reason = "No irrigation required under current conditions."

    return {
        "needs_irrigation": needs,
        "recommended_liters": round(water_liters, 1),
        "reason": reason
    }

model = load_or_train_model()

@app.route("/")
def index():
    return render_template("index.html")

@app.route("/api/health")
def health():
    return jsonify({"status": "ok"})

@app.route("/api/sensors")
def sensors():
    if not os.path.exists(DATA_FILE):
        generate_seed_data()

    df = pd.read_csv(DATA_FILE).tail(10)
    return jsonify(df.to_dict(orient="records"))

@app.route("/api/predict", methods=["POST"])
```

Edit with WPS Office

```
def predict():
    data = request.get_json() or {}

    sample = {
        "soil_moisture": float(data.get("soil_moisture", 30)),
        "temperature": float(data.get("temperature", 28)),
        "humidity": float(data.get("humidity", 50)),
        "rainfall": float(data.get("rainfall", 5)),
        "crop_stage": int(data.get("crop_stage", 2))
    }

    model_prediction = bool(model.predict(pd.DataFrame([sample]))[0])
    plan = calculate_irrigation_plan(sample)

    final_need = model_prediction and plan["needs_irrigation"]
    final_liters = plan["recommended_liters"] if final_need else 0.0

    if not final_need:
        final_reason = "Irrigation avoided to save water and reduce waste."
    else:
        final_reason = plan["reason"]

    result = {
        "needs_irrigation": final_need,
        "recommended_liters": round(final_liters, 1),
        "confidence": round(float(max(model.predict_proba(pd.DataFrame([sample]))[0])), 2),
        "reason": final_reason,
        "input": sample
    }
    return jsonify(result)

if __name__ == "__main__":
    app.run(debug=True, host="0.0.0.0", port=5000)
    </body>
</html>
@app.route("/api/sensors")
def sensors():
    if not os.path.exists(DATA_FILE):
        generate_seed_data()

    df = pd.read_csv(DATA_FILE).tail(10)
    return jsonify(df.to_dict(orient="records"))
    <h2>Recent Sensor Data</h2>
<table border="1" cellpadding="8" cellspacing="0" style="width:100%; border-collapse:collapse;">
  <thead>
    <tr style="background:#dfeedd;">
      <th>Soil Moisture</th>
      <th>Temperature</th>
      <th>Humidity</th>
      <th>Rainfall</th>
      <th>Crop Stage</th>
      <th>Irrigation</th>
    </tr>
  </thead>
  <tbody id="sensorTableBody"></tbody>
</table>

<script>
async function loadSensors() {
    const res = await fetch("/api/sensors");
```

Edit with WPS Office

```
        const data = await res.json();

        const tableBody = document.getElementById("sensorTableBody");
        tableBody.innerHTML = "";

        data.forEach(row => {
            const tr = document.createElement("tr");
            tr.innerHTML = `
                <td>${row.soil_moisture}%</td>
                <td>${row.temperature}捞 C</td>
                <td>${row.humidity}%</td>
                <td>${row.rainfall} mm</td>
                <td>${row.crop_stage}</td>
                <td>${row.needs_irrigation ? "Yes" : "No"}</td>
            `;
            tableBody.appendChild(tr);
        });
}

loadSensors();
setInterval(loadSensors, 3000);
</script>
@app.route("/api/sensors")
def sensors():
        if not os.path.exists(DATA_FILE):
            generate_seed_data()

        df = pd.read_csv(DATA_FILE).tail(10)
        return jsonify(df.to_dict(orient="records"))
        <h2>Recent Sensor Data</h2>
<table border="1" cellpadding="8" cellspacing="0" style="width:100%; border-collapse:collapse;">
   <thead>
      <tr style="background:#dfeedd;">
         <th>Soil Moisture</th>
         <th>Temperature</th>
         <th>Humidity</th>
         <th>Rainfall</th>
         <th>Crop Stage</th>
         <th>Irrigation</th>
      </tr>
   </thead>
   <tbody id="sensorTableBody"></tbody>
</table>

<script>
async function loadSensors() {
        const res = await fetch("/api/sensors");
        const data = await res.json();

        const tableBody = document.getElementById("sensorTableBody");
        tableBody.innerHTML = "";

        data.forEach(row => {
            const tr = document.createElement("tr");
            tr.innerHTML = `
                <td>${row.soil_moisture}%</td>
                <td>${row.temperature}捞 C</td>
                <td>${row.humidity}%</td>
                <td>${row.rainfall} mm</td>
                <td>${row.crop_stage}</td>
```

Edit with WPS Office

```
                <td>${row.needs_irrigation ? "Yes" : "No"}</td>
            `;
            tableBody.appendChild(tr);
        });
}

loadSensors();
setInterval(loadSensors, 3000);
</script>
def get_crop_condition(sample):
    soil = sample["soil_moisture"]
    temp = sample["temperature"]
    humidity = sample["humidity"]
    rainfall = sample["rainfall"]

    score = 0

    if soil < 25:
        score -= 3
    elif soil < 40:
        score -= 1
    else:
        score += 2

    if temp > 35:
        score -= 3
    elif temp > 30:
        score -= 1
    else:
        score += 1

    if humidity < 35:
        score -= 3
    elif humidity < 55:
        score -= 1
    else:
        score += 2

    if rainfall > 15:
        score += 2
    elif rainfall > 5:
        score += 1
    else:
        score -= 1

    if score >= 3:
        condition = "Healthy"
    elif score >= 0:
        condition = "Moderate"
    else:
        condition = "Critical"

    return {
        "condition": condition,
        "score": score
    }
        crop_status = get_crop_condition(sample)

    result = {
        "needs_irrigation": bool(prediction),
```

Edit with WPS Office

```
        "confidence": round(confidence, 2),
        "crop_condition": crop_status["condition"],
        "crop_score": crop_status["score"],
        "input": sample
    }
    <th>Crop Condition</th>
    <td>${row.crop_condition || "Unknown"}</td>
    document.getElementById("result").innerHTML = `
    <strong>${status}</strong><br>
    Crop Condition: ${data.crop_condition}<br>
    Crop Score: ${data.crop_score}<br>
    Confidence: ${(data.confidence * 100).toFixed(1)}%<br>
    Soil moisture: ${data.input.soil_moisture}%<br>
    Temperature: ${data.input.temperature}掳 C
`;
        if soil_moisture < 25 or humidity < 35:
            crop_condition = "Critical"
        elif soil_moisture < 40 or humidity < 55:
            crop_condition = "Moderate"
        else:
            crop_condition = "Healthy"

        rows.append({
            "soil_moisture": soil_moisture,
            "temperature": temperature,
            "humidity": humidity,
            "rainfall": rainfall,
            "crop_stage": crop_stage,
            "needs_irrigation": needs_irrigation,
            "crop_condition": crop_condition
        })
        def get_crop_condition(sample):
    soil = sample["soil_moisture"]
    temp = sample["temperature"]
    humidity = sample["humidity"]
    rainfall = sample["rainfall"]

    score = 0

    if soil < 25:
        score -= 3
    elif soil < 40:
        score -= 1
    else:
        score += 2

    if temp > 35:
        score -= 3
    elif temp > 30:
        score -= 1
    else:
        score += 1

    if humidity < 35:
        score -= 3
    elif humidity < 55:
        score -= 1
    else:
        score += 2
```

Edit with WPS Office

```
if rainfall > 15:
        score += 2
elif rainfall > 5:
        score += 1
else:
        score -= 1

if score >= 3:
        condition = "Healthy"
elif score >= 0:
        condition = "Moderate"
else:
        condition = "Critical"

return {
        "condition": condition,
        "score": score
}
```

Edit with WPS Office

AI Smart Agriculture

AI Smart Agriculture is the use of Artificial Intelligence, sensors, and data analysis to make farming more efficient, intelligent, and sustainable.

The system collects important agricultural data such as soil moisture, temperature, humidity, rainfall, and crop growth stage. AI analyzes these conditions and predicts whether irrigation is required.

The proposed system uses a Machine Learning model to support irrigation decisions. Based on the available sensor data, it can recommend whether water should be supplied to the crops. The system also considers rainfall and soil conditions to avoid unnecessary irrigation and reduce water wastage.

This technology can help farmers save water, improve crop management, reduce manual monitoring, and make better decisions based on data. The system can also classify crop conditions as Healthy, Moderate, or Critical based on environmental factors.

Source: AI Smart Agriculture project PDF provided by the user.

AI Smart Agriculture – Project Explanation

This project uses Artificial Intelligence (AI), sensor data, and Machine Learning to support smarter irrigation and crop monitoring. The main aim is to give farmers a data-based decision instead of depending only on manual observation.

1. How the System Works

The system takes five important inputs: soil moisture, temperature, humidity, rainfall, and crop growth stage. These values represent the environmental conditions around the crop. The AI model studies these inputs and predicts whether irrigation is required.

2. Machine Learning Model

The project uses a Random Forest Classifier. Training data is prepared using agricultural conditions, and the data is divided into training and testing sets. The model learns patterns between the sensor values and the irrigation requirement. It also produces a confidence value for its prediction.

3. Irrigation Decision

The system does not simply turn irrigation on whenever the soil is dry. It also checks rainfall, soil moisture, temperature, humidity, and crop stage. If rainfall is sufficient, irrigation can be skipped. This helps reduce unnecessary water usage and overwatering.

4. Recommended Water Quantity

The project contains an irrigation-plan calculation that estimates a recommended amount of water in litres. The calculation considers the soil moisture deficit, temperature load, humidity, rainfall, and a crop-stage factor. The result is limited to avoid an excessive recommendation.

5. Crop Condition Monitoring

The system can also classify crop condition as Healthy, Moderate, or Critical. A score is calculated from soil moisture, temperature, humidity, and rainfall. Better environmental conditions increase the score, while stressful conditions reduce it.

6. Dashboard

The web dashboard displays recent sensor readings and provides a form where the user can enter current conditions. After clicking Analyze, the system displays the irrigation decision, confidence, crop condition, crop score, and relevant input values. Recent sensor data is refreshed periodically.

7. Technologies Used

Python is used for the backend, Flask provides the web application and API routes, Pandas handles the sensor dataset, Scikit-learn provides the Random Forest Machine Learning model, Joblib is used to save/load the trained model, and HTML/CSS/JavaScript are used for the dashboard interface.

8. Benefits

The proposed system can help reduce manual monitoring, improve irrigation decisions, save water, and support better crop management. By combining sensor information with AI-based analysis, farming

decisions can become more efficient and data-driven.

9. Simple Example

For example, when soil moisture is low, temperature is high, humidity is low, and there has been little rainfall, the system may identify that irrigation is needed. If significant rainfall has already occurred, it can avoid irrigation to reduce water wastage.